



JavaScript Fundamental...



A 1.20

|A| 0%



What is a Promise in JavaScript? (JavaScript में एक प्रॉमिस क्या है?)

A Promise in JavaScript is an object that represents the eventual completion or failure of an asynchronous operation and its resulting value. It allows you to write asynchronous code that looks more like synchronous code, making it easier to read and manage than traditional callback-based approaches (avoiding callback hell).

A Promise can be in one of three states:

- Pending:** Initial state, neither fulfilled nor rejected.
- Fulfilled (Resolved):** The operation completed successfully.
- Rejected:** The operation failed.

Example:

```
const myPromise = new Promise((resolve, reject) => {
  // Simulate an asynchronous operation
  const success = true; // Simulate an asynchronous operation
  setTimeout(() => {
    if (success) {
      resolve("Data fetched successfully!");
    } else {
      reject("Error fetching data.");
    }
  }, 2000);
});

myPromise
  .then(message => {
    console.log("Success:", message); // Will log after 2 seconds if success is true
  })
  .catch(error => {
    console.error("Failure:", error); // Will log after 2 seconds if success is false
  });

console.log("Promise initiated (synchronous code continues)");
```

Hindi: JavaScript में एक प्रॉमिस एक ऑब्जेक्ट है जो एक एसिंक्रोनस ऑपरेशन के अंतर्गत पूरा होने या विफल होने और उसके परिणामस्वरूप मान का प्रतिनिधित्व करता है। यह आपको एसिंक्रोनस कोड लिखने की अनुमति देता है जो सिंक्रोनस कोड जैसा दिखता है, जिससे इसे पढ़ने और लिखने में आसानी होती है। प्रॉमिस तब तक अवस्था में रहता है जब तक कि ऑपरेशन में सफलता नहीं मिलती और तब तक कि त्रुटि नहीं आती।

- पीडिंग (Pending):** प्रारंभिक अवस्था, न तो पूरा हुआ और न ही अस्वीकृत।
- फुलफिल्ड (Fulfilled) (रिजल्ट):** ऑपरेशन सफलतापूर्वक पूरा हो गया।
- रिजेक्टेड (Rejected):** ऑपरेशन विफल हो गया।

उदाहरण:

```
const myPromise = new Promise((resolve, reject) => {
  const success = true; // एक एसिंक्रोनस ऑपरेशन का अनुकरण करें
  setTimeout(() => {
    if (success) {
      resolve("Data fetched successfully!");
    } else {
      reject("Error fetching data.");
    }
  }, 2000);
});

myPromise
  .then(message => {
    console.log("Success:", message); // यदि success true है तो 2 सेकंड के बाद सारा होगा
  })
  .catch(error => {
    console.error("Failure:", error); // यदि success false है तो 2 सेकंड के बाद सारा होगा
  });

console.log("Promise initiated (synchronous code continues)");
```

How to handle Promises in JavaScript (JavaScript में प्रॉमिस को कैसे हैंडल करें)

Promises are handled using `then()`, `catch()`, and `finally()` methods.

- `then(onFulfilled, onRejected):`**
 - `onFulfilled`: A callback function that runs if the promise is fulfilled. It receives the resolved value.
 - `onRejected`: An optional callback function that runs if the promise is rejected. It receives the reason for rejection (error). It's generally better to use `catch()` for error handling.
- `catch(onRejected):`**
 - A more readable way to handle rejections. It's syntactic sugar for `then(null, onRejected)`.
- `finally(onFinally):`**
 - A callback function that runs regardless of whether the promise was fulfilled or rejected. It's useful for cleanup operations.

Example:

```
function fetchData(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve("Data from server");
      } else {
        reject("Failed to fetch data");
      }
    }, 1000);
  });
}

// Handling a successful promise
fetchData(true)
  .then(data => console.log("Success (then):", data))
  .catch(error => console.error("Error (catch):", error))
  .finally(() => console.log("Promise operation finished."));

// Handling a rejected promise
fetchData(false)
  .catch(error => console.log("Success (then):", data))
  .catch(error => console.error("Error (catch):", error))
  .finally(() => console.log("Promise operation finished."));
```

Hindi: प्रॉमिस को `then()`, `catch()`, और `finally()` विधियों का उपयोग करके हैंडल किया जाता है।

- `then(onFulfilled, onRejected):`**
 - `onFulfilled`: एक कॉलबैक फंक्शन जो प्रॉमिस पूरा होने पर चलता है। यह रिजल्ट मान प्राप्त करता है।
 - `onRejected`: एक वैकल्पिक कॉलबैक फंक्शन जो प्रॉमिस अस्वीकृत होने पर चलता है। यह त्रुटि का कारण (एरर) प्राप्त करता है। इसे हैंडलिंग के लिए `catch()` का उपयोग करना और तब पर बेहतर होता है।
- `catch(onRejected):`**
 - संक्षिप्त विधि जो त्रुटि को एक अधिक पठनीय तरीका। यह `then(null, onRejected)` के लिए सिंक्रोनिक श्रृंखला है।
- `finally(onFinally):`**
 - एक कॉलबैक फंक्शन जो प्रॉमिस पूरा हुआ या अस्वीकृत हुआ, हमेशा निष्पत्ति दिया करता है। यह सफाई ऑपरेशन के लिए उपयोगी है।

उदाहरण:

```
function fetchData(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve("Data from server");
      } else {
        reject("Failed to fetch data");
      }
    }, 1000);
  });
}

// एक सफल प्रॉमिस को हैंडल करना
fetchData(true)
  .then(data => console.log("Success (then):", data))
  .catch(error => console.error("Error (catch):", error))
  .finally(() => console.log("Promise operation finished."));

// एक खराब प्रॉमिस को हैंडल करना
fetchData(false)
  .then(data => console.log("Success (then):", data))
  .catch(error => console.error("Error (catch):", error))
  .finally(() => console.log("Promise operation finished."));
```

What is Asynchronous Programming in JavaScript? (JavaScript में एसिंक्रोनस प्रोग्रामिंग क्या है?)

Asynchronous programming in JavaScript refers to the execution of tasks that don't run in a strict, sequential order. Instead of blocking the execution of the rest of the code while waiting for a long-running operation (like network requests, file I/O, or timers) to complete, asynchronous operations allow the program to continue executing other tasks. Once the asynchronous operation finishes, a callback or Promise handler is executed.

Common Asynchronous Operations:

- `setTimeout`, `setInterval`
- Network requests (e.g., `fetch`, `XMLHttpRequest`)
- File reading/writing
- Database operations

Mechanisms for Asynchronous JavaScript:

- Callbacks
- Promises (ES6)
- `async/await` (ES2017, built on Promises)

Example:

```
function fetchData(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve("Data from server");
      } else {
        reject("Failed to fetch data");
      }
    }, 1000);
  });
}

// एक सफल प्रॉमिस को हैंडल करना
fetchData(true)
  .then(data => console.log("Success (then):", data))
  .catch(error => console.error("Error (catch):", error))
  .finally(() => console.log("Promise operation finished."));

// एक खराब प्रॉमिस को हैंडल करना
fetchData(false)
  .then(data => console.log("Success (then):", data))
  .catch(error => console.error("Error (catch):", error))
  .finally(() => console.log("Promise operation finished."));
```

Hindi: टाइप ऑपरेशन एक स्ट्रिंग लीटा है जो अनिश्चित ऑपरेशन के पूरा होने तक रुकता है। यह एक ऑपरेशन या मान के डेटा तक की राह को ब्लॉक करता है।

समानांतर रिटर्न मान:

- `"undefined"`
- `"boolean"`
- `"number"`
- `"string"`
- `"symbol"`
- `"bigint"`
- `"object"` (ऑब्जेक्ट, एरर और null के लिए, एक क्लास तक)
- `"function"` (फंक्शन के लिए)

उदाहरण:

```
const fruits = ["apple", "banana", "cherry"];
const colors = ["red", "green", "blue"];

function fetchData(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve("Data from server");
      } else {
        reject("Failed to fetch data");
      }
    }, 1000);
  });
}
```

How to add an element to an array in JavaScript? (JavaScript में एक एर्रे में एक एलिमेंट कैसे जोड़ें)

You can add elements to an array using various methods:

- `push()`:** Adds one or more elements to the **end** of an array and returns the new length.

```
let fruits = ["apple", "banana"];
fruits.push("cherry");
console.log(fruits); // ["apple", "banana", "cherry"]
```
- `unshift()`:** Adds one or more elements to the **beginning** of an array and returns the new length.

```
let numbers = [2, 3];
```

```
console.log("Start");

setTimeout(() => {
  console.log("Async operation completed after 2 seconds.");
}, 2000);

fetch("https://api.example.com/data") // Simulating a network request
  .then(response => response.json())
  .then(data => console.log("Async data fetched:", data))
  .catch(error => console.error("Fetch error:", error));

console.log("End (This runs before async operations complete)");
```

Hindi: JavaScript में एसिंक्रोनस प्रोग्रामिंग उन कार्यों के निष्पादन को संभालती है जो जोड़ सकते हैं, अनुक्रमिक क्रम में नहीं करते हैं। एक ऐसा काम पूरा करने का ऑपरेशन (जैसे नेटवर्क अनुरोध, फाइल I/O, या टाइमर) के पूरा होने की प्रतीक्षा करते हुए बाकी कोड के निष्पादन को ब्लॉक करने के बजाय, एसिंक्रोनस ऑपरेशन प्रोग्राम को अन्य कार्यों को निष्पादन करना जारी रखने की अनुमति देता है। एक बार जब एसिंक्रोनस ऑपरेशन समाप्त हो जाता है, तो एक कॉलबैक या प्रॉमिस हैंडलर निष्पत्ति देता है।

सामान्य एसिंक्रोनस ऑपरेशन:

- `setTimeout`, `setInterval`
- नेटवर्क अनुरोध (उदाहरण के लिए `fetch`, `XMLHttpRequest`)
- फाइल पढ़ना/लिखना
- डेटाबेस ऑपरेशन

एसिंक्रोनस JavaScript के लिए स्पेसिफिकेशन:

- कॉलबैक
- प्रॉमिस (ES6)
- `async/await` (ES2017, प्रॉमिस पर निर्भर)

उदाहरण:

```
console.log("Start");

setTimeout(() => {
  console.log("Async operation completed after 2 seconds.");
}, 2000);

fetch("https://api.example.com/data") // एक नेटवर्क अनुरोध का अनुकरण करना
  .then(response => response.json())
  .then(data => console.log("Async data fetched:", data))
  .catch(error => console.error("Fetch error:", error));

console.log("End (यह एसिंक्रोनस ऑपरेशन पूरा होने से पहले चलता है)");
```

Explain the Event Loop in JavaScript (JavaScript में इवेंट लूप समझाएं)

The Event Loop is a fundamental concept in JavaScript's concurrency model that allows JavaScript to perform non-blocking I/O operations despite being single-threaded.

Key Components:

- Call Stack:** Where synchronous code is executed. Functions are pushed onto the stack and popped off when they return.
- Web APIs / Node.js APIs:** Provided by the browser or Node.js runtime (not JavaScript itself) for asynchronous tasks like `setTimeout`, `DOM events`, `fetch requests`. When these tasks complete, their callbacks are moved to the Task Queue.
- Task Queue (Callback Queue / Microtask Queue):** Stores callbacks/functions from Web APIs (e.g., `setTimeout`, `DOM events`).
- Microtask Queue:** A higher-priority queue for callbacks from `Promises` (`then()`, `catch()`), `finally()`, `queueMicrotask`, `MutationObserver`. Microtasks are processed before macrotasks in each event loop iteration.
- Event Loop:** Continuously monitors the Call Stack and the Task Queues. If the Call Stack is empty, it takes the first function from the Microtask Queue and pushes it onto the Call Stack. If the Microtask Queue is empty, it takes the first function from the Task Queue and pushes it onto the Call Stack.

How it works:

- Synchronous code runs to completion on the Call Stack.
- When an asynchronous operation is encountered, it's handed off to a Web API.
- The JavaScript engine continues executing synchronous code.
- When the Web API finishes its task, it places the associated callback into the appropriate queue (Microtask or Task).
- The Event Loop then picks up callbacks from the queues and pushes them onto the Call Stack when the Call Stack is empty.

Example:

```
let numbers = [2, 3];
numbers.push(4);
console.log(numbers); // [1, 2, 3]

// 3. रॉड सिस्टम ( ) ... जोड़ें यह एलिमेंट के साथ एक नया एर्रे बनाएं।
let colors = ["red", "green"];
colors = [...colors, "blue"]; // अब नए जोड़ें
console.log(colors); // ["red", "green", "blue"]

colors = ["yellow", ...colors]; // रूपांतरण में जोड़ें
console.log(colors); // ["yellow", "red", "green", "blue"]

// 4. डायरेक्ट असाइनमेंट (यदि आपको इच्छा है):
let arr = [1, 2];
arr[2] = 3; // इंडेक्स 2 पर जोड़ें
console.log(arr); // [1, 2, 3]

arr[3] = 4; // यहाँ इच्छा रखेंगे जोड़ें से परे है तो
console.log(arr); // [1, 2, 3, <2 empty items>, 4]

// 5. splice(): किसी भी स्थिति में एलिमेंट्स जोड़ें या हटा सकते हैं।
let veggies = ["carrot", "beet"];
let index = splice(startIndex, deleteCount, item, ...items);
veggies.splice(1, 0, "cabbage"); // इच्छा 1 पर, 0 आइटम हटाएं, "cabbage" जोड़ें
console.log(veggies); // ["carrot", "cabbage", "beet"]
```

What is JSON and how do you parse it in JavaScript? (JSON क्या है और आप इसे JavaScript में कैसे पार्स करते हैं?)

JSON (JavaScript Object Notation) is a lightweight data interchange format. It is human-readable and easy for machines to parse and generate. JSON is often used when data is sent from a server to a web page, or vice versa. It is based on a subset of the JavaScript programming language.

Key characteristics:

- Data is in `name=value` pairs.
- Data is separated by commas.
- Curly braces `{}` hold objects.
- Square brackets `[]` hold arrays.

Parsing JSON in JavaScript: JavaScript provides a built-in JSON object with methods to work with JSON:

- `JSON.parse()`: Converts a JSON string into a JavaScript object.
- `JSON.stringify()`: Converts a JavaScript object into a JSON string.

Example:

```
// A JSON string
const jsonString = '{"name": "Alice", "age": 30, "isStudent": false, "courses": ["Math"]}'

// Parse it into a JavaScript object
const jsonObj = JSON.parse(jsonString);
console.log(jsonObj);
```

```
console.log("4. End");
// अंतिम अद्युत्तर:
```

```
baseline
console.log(colors); // ["yellow", "red", "green", "blue"]
```

```
};
const jsonObjString =
JSON.stringify(anotherObject);
console.log(jsonObjString); // {"city": "New
```

Edit with chat